Documentación completa 2 5 OpenAPI y Orval

Integración backend y frontend City Care | Versión 1.0

Resultado. El apartado 2.5 está terminado. El contrato OpenAPI se conserva como fuente versionable, Orval genera las 16 operaciones del backend dentro del frontend Nuxt y un cliente HTTP común resuelve la URL del entorno, el Bearer, las respuestas JSON y los errores de la API.

Estado	Cobertura	Evidencia
COMPLETO	6 de 6 puntos del apartado 2.5	SDK con 16 operaciones
VALIDADO	Orval, TypeScript y build Nuxt	Tres comandos exitosos
INTEGRADO	Login, perfil y logout	Sesión y Bearer centralizados

1 Alcance terminado

Punto	Implementación	Estado
Swagger	Contrato disponible en /swagger e interfaz en /docs	Listo
Cliente Orval	16 funciones y tipos generados en el frontend	Listo
Bearer centralizado	cityCareFetch agrega Authorization automáticamente	Listo
URL por entorno	NUXT_PUBLIC_API_BASE_URL mediante runtimeConfig	Listo
Regeneración	npm run openapi:generate usa el contrato local	Listo
Frontend real	SDK y sesión integrados en mi-proyecto	Listo

2 Flujo de generación

El backend es la autoridad del contrato. La copia versionada openapi/city-care.v1.yaml se coloca en el frontend y Orval transforma sus esquemas, parámetros, cuerpos y respuestas en TypeScript. El archivo generado no se edita manualmente.

```
backend: openapi/city-care.v1.yaml
frontend: openapi/city-care.v1.yaml
npm run openapi:generate
salida: app/generated/api/city-care.ts
```

3 Operaciones generadas

Grupo	Funciones
Autenticación	login, getProfile, logout
Dependencias	list, create, get, update, updateStatus, updateManagers
Roles	listRoles
Usuarios	list, create, get, update, updateStatus, resetPassword

4 Cliente HTTP central

Orval utiliza app/services/api/api.client.ts como mutator. Todas las funciones pasan por cityCareFetch, por lo que ninguna vista necesita repetir la lectura del token ni construir manualmente encabezados de autenticación.

- Lee la cookie city-care-token al ejecutar cada solicitud.
- Agrega Authorization: Bearer cuando existe una sesión.
- Solicita y procesa JSON con Accept: application/json.
- Conserva credentials: include para escenarios con cookies.
- Convierte respuestas no exitosas en CityCareApiError con status y error de la API.

5 Configuración por entorno

Nuxt expone apiBaseUrl mediante runtimeConfig.public. El valor se obtiene de NUXT_PUBLIC_API_BASE_URL, por lo que el mismo código puede utilizar desarrollo, pruebas, staging o producción sin volver a generar el SDK.

Entorno	Ejemplo
Desarrollo	NUXT_PUBLIC_API_BASE_URL=http://localhost:3333
Staging	NUXT_PUBLIC_API_BASE_URL=https://api-staging.ejemplo.mx
Producción	NUXT_PUBLIC_API_BASE_URL=https://api.ejemplo.mx

6 Ciclo de sesión

Momento	Comportamiento
Login	Guarda token y usuario; dirige al panel según el rol
Recarga	El plugin auth.client restaura el perfil si existe token
Solicitud protegida	El mutator agrega el Bearer sin intervención del componente
Token inválido	La restauración limpia la sesión local
Logout	Revoca el token en el backend y limpia el estado local

7 Archivos principales

Archivo	Responsabilidad
openapi/city-care.v1.yaml	Contrato versionado del backend
orval.config.ts	Entrada, salida, cliente fetch y mutator
app/generated/api/city-care.ts	Tipos y funciones generadas
app/services/api/api.client.ts	Bearer y manejo uniforme de errores
app/services/auth/auth.service.ts	Login, perfil y logout
app/composables/useAuth.ts	Estado, persistencia y restauración
app/plugins/auth.client.ts	Restauración al iniciar el navegador
nuxt.config.ts	URL pública por entorno

8 Validación técnica

Comprobación	Resultado
npm run openapi:generate	Orval 8.46 genera 16 operaciones
npm run typecheck	Nuxt y vue-tsc sin errores
npm run build	Build de producción exitoso
Respuesta local	Frontend y backend responden HTTP 200
npm audit	Sin vulnerabilidades altas ni críticas; quedan avisos moderados transitivos

9 Uso para el equipo

- Actualizar primero el contrato OpenAPI cuando cambie una ruta del backend.
- Copiar el contrato actualizado al frontend y ejecutar npm run openapi:generate.
- No modificar app/generated/api/city-care.ts manualmente.
- Consumir las funciones generadas desde servicios o composables de cada módulo.
- Ejecutar typecheck y build antes de integrar cambios.

10 Conclusión

El apartado 2.5 queda cerrado. Backend y frontend comparten un contrato verificable, la autenticación se adjunta desde un único punto y el SDK puede regenerarse de forma repetible. El siguiente bloque es 2.6: conectar los paneles visuales con las operaciones ya disponibles y proteger las rutas por rol.